

# Listing\_Platform\_Technology

## AppExchange Listing — “If your solution contains Salesforce Platform technology”

**Field on the Partner Console submission form:** *If your solution contains Salesforce Platform technology, such as Lightning Components and Apex, provide details.*

**Submitted for:** Portwood DocGen Managed v2.1.0 (04tVx000000Zw5xIAC), 2026-05-22 re-submission **Companion attachment:** DocGen\_Platform\_Technology.md (this folder) is the long-form version of this field response. Either may be attached or pasted; this Listing\_\* version is form-ready prose.

---

### Paste-ready text

Portwood DocGen is built entirely on Salesforce Platform technology. All business logic runs in Apex, all admin UI is implemented in Lightning Web Components, the guest-user signing experience uses Visualforce pages hosted on a Salesforce Site, and asynchronous work is orchestrated via Platform Events, Queueable Apex, Batchable Apex, and Schedulable Apex. There are no external services, no callouts, no Named Credentials, and no third-party JavaScript or Apex libraries.

Package type: **Managed 2GP** Namespace: **portwoodglobal** API version: **66.0**

### Apex

#### Entry-point annotations used

- @AuraEnabled — for all LWC-invocable controllers
- @AuraEnabled(cacheable=true) — for read-only @wire endpoints
- @RemoteAction — used alongside @AuraEnabled on guest-user methods so Visualforce pages can invoke the same methods via JavaScript remoting
- @InvocableMethod — for Flow-callable actions (DocGenFlowAction, DocGenBulkFlowAction, DocGenGiantQueryFlowAction, DocGenSignatureFlowAction)

- `@future(callout=false)` — *not used* (no callouts)

## Apex patterns and class counts (v2.1.0)

- **Controllers (8 classes)** — DocGenController, DocGenBulkController, DocGenChartImageController, DocGenSetupController, DocGenSignatureController, DocGenSignatureSenderController, DocGenAuthenticatorController, DocGenTemplateManager.
- **Service / merge-engine classes (~26 classes)** — DocGenService (the merge engine, ~10K lines), DocGenDataRetriever (V1/V2/V3 SOQL execution), DocGenHtmlRenderer (OOXML→HTML conversion for `Blob.toPdf()`), DocGenGiantQueryAssembler + DocGenGiantQueryBatch + DocGenGiantQueryStitchJob (memory-safe large-relationship rendering), the v1.99 chart engine pipeline (DocGenChartRasterizer, DocGenChartTagExpander, DocGenChartBucketResolver, DocGenChartFont, DocGenPngEncoder, DocGenSvgChartSerializer), BarcodeGenerator (Code-128 / QR), DocGenSignatureService (signature stamping + PDF queueable), DocGenSignatureEmailService (branded signing emails), DocGenApprovalHistory, DocGenException, DocGenSignatureGuestSecurity (shipped in v2.0 — guest-context CRUD/FLS Schema-describe-check helper), and DocGenFlsGuard (NEW in v2.1.0 — per-field FLS describe-check helper; `assertCreateable/assertUpdateable/assertAccessible`; 243 call sites across 19 controllers).
- **Async Apex**
  - **Queueable** — chart image rasterization, template image extraction on save (DocGenMergeJob), DocGenSignatureService.SignaturePdfQueueable + TemplateSignaturePdfQueueable for post-signature PDF generation and email delivery
  - **Batchable** — DocGenBatch for generating thousands of documents across a saved query; DocGenGiantQueryBatch for multi-million-row child datasets
  - **Schedulable** — DocGenSignatureReminderSchedulable runs hourly when admin enables reminders
- **Triggers** — one Apex trigger (DocGenSignaturePdfTrigger on the DocGen\_Signature\_PDF\_\_e platform event) — runs in Automated Process user context, bridges guest-user limitations (no User-table access, no session ID, no email sending) to system-context work.
- **Test classes (29 classes)** — 1,449 test methods, org-wide code coverage 76%, enforced pre-release. Every release run also passes 11 end-to-end anonymous Apex scripts (scripts/e2e-01 through e2e-08 + four e2e-07-syntax\* syntax suites).

## Sharing and security (v2.1.0)

- All controllers use either `with sharing` or `without sharing` explicitly (never the implicit default). `without sharing` is used only where required for guest-user signing or system-context post-signature processing, and each such class has an inline justification documenting the cryptographic token gating.
- Every admin `@AuraEnabled` / `@InvocableMethod` entry point opens with an explicit `Schema.sObjectType.<Object>.isAccessible | isCreateable | isUpdateable()` gate (shipped in v2.0). This is the documented enforcement signal the `sfgc:ApexFlsViolation` rule pattern-matches on, and it is the reviewer's stated first alternative ("enforce CRUD checks on the object... **or** alternatively use `USER_MODE`") from the v1.56 finding-resolution language.

- v2.1.0 adds **per-field**  
**Schema.SObjectField.getDescribe().is{Createable,Updateable,Accessible}()** checks via the new **DocGenFlsGuard helper** immediately before every DML and WITH SYSTEM\_MODE SOQL statement. 243 call sites across 19 controllers (70 DML guard sites across 9 controllers + 173 SOQL guard sites across 18 classes). This implements the AND half of the reviewer's stated finding-resolution language ("enforce CRUD checks on the object AND FLS checks on the fields before performing any DML operation") line-for-line.
- Standard-object SOQL (ContentVersion, ContentDocumentLink, User, OrgWideEmailAddress, etc.) runs WITH USER\_MODE throughout. Package-namespaced custom-object SOQL/DML runs WITH SYSTEM\_MODE behind both the Schema-CRUD gate AND the DocGenFlsGuard per-field check, with `/* code-analyzer-suppress ApexFlsViolation */` and inline justification — required because USER\_MODE strict-FLS strips package-namespaced custom fields when subscriber admin profiles haven't been granted FLS individually per release (reproducible in package-build orgs).
- Guest signing endpoints route through DocGenSignatureGuestSecurity (shipped in v2.0) for Schema-CRUD describe checks + 64-char SHA-256 hex token-shape validation + per-operation field allowlists, all documented inline at every call site. v2.1.0 layers DocGenFlsGuard per-field guards on top.
- All dynamic field and object references are validated against `Schema.getGlobalDescribe()` or `Schema.describeSObjects()` field maps before assembly into SOQL. User-supplied values pass through `bind variables` or `String.escapeSingleQuotes()`. WHERE/ORDER BY clauses pass through a keyword blocklist (INSERT, UPDATE, DELETE, DROP, ALTER, GRANT, SELECT, ;, --, /\*).

## Lightning Web Components

**18 LWC bundles**, all namespaced, all Lightning Web Security (LWS) compatible. No Aura components.

### Primary components

- `docGenCommandHub` — main app tab; welcome banner, quick-action cards, embedded template manager, embedded bulk runner, Learning Center
- `docGenAdmin` — template CRUD, version management, template wizard (query builder + test record picker); wired to the v1.99 chart engine and v3 signature placement builder
- `docGenColumnBuilder` — visual multi-object query tree builder (V3 query format)
- `docGenQueryBuilder` — legacy flat query builder, retained for V1/V2 configs
- `docGenTreeBuilder` + `docGenTreeNode` — recursive query-tree builder (the tree node component is the one flagged by `pmd:AvoidLwcBubblesComposedTrue` — `composed: true` is structurally required because each level lives in its own shadow DOM and events must reach the root)
- `docGenBulkRunner` — bulk generation launcher with saved-query picker and progress tracking
- `docGenRunner` — record-page component for generating a document from a single record. Includes `docGenZipWriter.js` — a pure-JavaScript ZIP writer (store mode, CRC-32 inline) used to assemble DOCX files client-side from server-rendered XML parts, bypassing the Apex 6 MB heap limit

- `docGenSignatureSender` — record-page component for creating signature requests; multi-template selection, signer row editor, preview modal, signing-order toggle (Sequential vs Parallel)
- `docGenSignatureSettings` — admin settings page with setup validation checklist, OWA selector, and reminder configuration
- `docGenAuthenticator` — public document verifier (drag-and-drop signed PDF → SHA-256 hashed in browser → audit lookup). v2.0 returns ALL signers for a multi-signer document (was LIMIT 1).
- Supporting components for template preview, job history, settings panels, page setup, sharing, and the column/filter/title editors

**v2.0 clickjacking remediation (carried into v2.1.0)** — all `style="position: absolute|fixed"` inline attributes on exposed LWCs replaced with the SLDS `slds-is-absolute` utility class + named CSS classes. Five bundles touched (`docGenAdmin`, `docGenAuthenticator`, `docGenBulkRunner`, `docGenQueryBuilder`, plus `docGenColumnBuilder` proactively). New `docGenAuthenticator.css` created.

## LWC patterns

- `@api` and `@track` used idiomatically
- `@wire` with imperative `refreshApex()` where refresh is needed
- `publish / subscribe` via `lightning/messageService` for cross-component communication (not for server events)
- All server calls go through `@AuraEnabled Apex` — no direct `fetch()` calls
- All binary data (image base64, file blobs) is returned from Apex, since LWS blocks client-side fetches to `/sfc/servlet.shepherd/` URLs

## Visualforce Pages

**4 Visualforce pages.** VF is used *only* for the guest-user e-signature experience (which requires a Salesforce Site) and the public document verifier.

- `DocGenSign.page / DocGenSignature.page` — primary signing UI, served to guest users via Site. Same `DocGenSignatureGuestSecurity` Schema-gates as the LWC path.
- `DocGenVerify.page` — public signature verification page (drag-and-drop signed PDF or `?id=<requestId>` parameter, shows full audit data with all signers in v2.0)
- `DocGenGuide.page` — in-app admin guide

All VF pages declare explicit controller classes and use `<apex:form>` + JavaScript Remoting to invoke Apex. URL parameters are validated against strict format patterns (e.g. `[a-fA-F0-9]{64}` for tokens, `[a-zA-Z0-9]{15,18}` for IDs) before reflection. The signing pages are served from the namespaced Site only.

## Platform Events

Two platform events, both internal to the package:

- **`DocGen_Signature_PDF__e`** — published by guest users (via `EventBus.publish()`) on signature completion, consumed by `DocGenSignaturePdfTrigger` in Automated Process context. Bridges guest-user limitations to system-context work.

- **DocGen\_Guest\_Render\_\_e** — published by guest-Experience-Cloud render contexts to defer expensive document rendering to a system-context Queueable.

No outbound platform event publishing to external subscribers. No Change Data Capture, no External Events.

## Custom Metadata, Objects, and Settings

- **11 custom objects** (9 sObjects + 2 platform events), all namespaced:
  - DocGen\_Template\_\_c, DocGen\_Template\_Version\_\_c, DocGen\_Saved\_Query\_\_c — template authoring
  - DocGen\_Job\_\_c — bulk generation tracking
  - DocGen\_Signature\_Request\_\_c, DocGen\_Signer\_\_c, DocGen\_Signature\_Placement\_\_c (NEW post-v1.56 — per-tag guided signing), DocGen\_Signature\_Audit\_\_c — signature pipeline
  - DocGen\_Settings\_\_c — hierarchical custom setting
  - DocGen\_Signature\_PDF\_\_e, DocGen\_Guest\_Render\_\_e — platform events
- **1 hierarchical custom setting** — DocGen\_Settings\_\_c — for org-level configuration (OWA ID, site URL, branding, reminder toggle). No protected metadata or protected settings are used.
- **No custom metadata types** — configuration is per-org via the custom setting, not packaged.

## Flow Integration

Four @InvocableMethod entry points:

- DocGenFlowAction — generate a document from a Flow step
- DocGenBulkFlowAction — bulk generation against a saved query
- DocGenGiantQueryFlowAction — multi-million-row query async generation; auto-detects when to switch to async based on child-relationship row counts
- DocGenSignatureFlowAction — create a signature request from a Flow step; accepts a typed Signer Apex-defined type for the input list

All four return result objects (never throw uncaught exceptions) so Flow builders can handle errors in a Decision element.

## Lightning App Builder Surfaces

- One App Page (the DocGen Command Hub tab)
- Record Page components (docGenRunner, docGenSignatureSender) — admins drag these onto any standard or custom object record page
- Flow Screen components on a subset (chart preview, query builder embed)
- No Home Page components, no App Page templates exposed for subscriber customization

## Permission Sets

**4 permission sets**, all packaged:

- `DocGen_Admin` — full CRUD/FLS on all custom objects, access to all VF pages, all Apex classes, all LWC tabs
- `DocGen_User` — read + edit on most objects (no delete on audit), access to user-facing VF pages, document generation and signature sender
- `DocGen_Guest_Runner` — read templates + render documents in guest-Experience-Cloud contexts. No write access to template metadata. Used by Experience Cloud guest portals offering self-service document generation.
- `DocGen_Guest_Signature` — read-only on signature objects through token-gated entry points only (now additionally gated by `DocGenSignatureGuestSecurity`); insert on `DocGen_Signature_Audit__c`; access to `DocGenSignature`, `DocGenSign`, `DocGenVerify` VF pages. Assigned to the Site's Guest User profile.

No permission set groups, no muting permission sets, no profile changes.

## Salesforce Features Explicitly Used

- `Blob.toPdf()` (Spring '26 Visualforce PDF Rendering Service — the Release Update is required and is documented in the post-install guide)
- `Messaging.SingleEmailMessage` with `setOrgWideEmailAddressId()` for guest-user-originated emails
- `Crypto.generateAesKey()`, `Crypto.generateDigest()`, `Crypto.getRandomInteger()` for token and PIN generation
- `Auth.SessionManagement.getCurrentSession()` for IP capture — guarded so it is never called in Guest context (throws uncatchable error there)
- `ContentVersion` for template storage and generated-document storage; `ContentDocumentLink` for record attachment; `ContentDistribution` for guest-rendered preview images
- `Database.query(String, AccessLevel)` with explicit `USER_MODE` or `SYSTEM_MODE` on every dynamic query
- `Database.insert/update/delete/upsert(record, AccessLevel)` with explicit modes on every DML
- `Schema.sObjectType.<Object>.isAccessible|isCreateable|isUpdateable|isDeletable()` describe checks at every admin and guest entry point
- `Security.stripInaccessible()` in some read paths where `USER_MODE` SOQL returns fields the caller may not have FLS on

## Features Explicitly Not Used

- No callouts, no Named Credentials, no Remote Site Settings
- No external JavaScript libraries in LWC (no npm imports, no CDN scripts)
- No Omni-Channel, OmniStudio, Einstein, or Service Console APIs
- No Experience Cloud (Community / Site.com) pages owned by the package — only the classic Salesforce Site for guest signing. Customers can host the LWC `docGenRunner` on their own Experience Cloud sites if they choose.
- No Change Data Capture, no External Events, no outbound messages
- No Canvas Apps, no Connected Apps (beyond the standard subscriber install)

- No managed package extensions of standard AppExchange apps
- 

## What changed since the v1.56 listing baseline

- **Class counts:** 27 → 42 non-test, 19 → 29 test (846 → 1,449 tests at 75% → 76% coverage).
  - **LWC bundles:** 17 → 18.
  - **Custom objects:** 9 → 11 (DocGen\_Signature\_Placement\_\_c for v3 guided signing, DocGen\_Guest\_Render\_\_e platform event for async guest render).
  - **Permission sets:** 3 → 4 (added DocGen\_Guest\_Runner for guest-Experience-Cloud document rendering).
  - **SOQL/DML execution mode:** v2.0 shipped the hybrid Schema-CRUD-gate + SYSTEM\_MODE pattern across every admin endpoint, replacing the v1.42/v1.99 “permission sets are the CRUD/FLS boundary” rationalization that the AppExchange reviewer rejected against the v1.56 listing.
  - **v2.1.0 per-field FLS guard layer:** new DocGenFlsGuard helper class performs explicit `Schema.SObjectField.getDescribe().is{Createable,Updateable,Accessible}()` checks at every admin DML and WITH SYSTEM\_MODE SOQL site. 243 guard call sites across 19 controllers (70 DML + 173 SOQL). Closes the 562 Checkmarx findings (FLS Create 118 + FLS Update 104 + USER\_MODE Missing 340) returned on the v2.0 source scan.
  - **Guest signing:** DocGenSignatureGuestSecurity helper class (shipped in v2.0) centralizes Schema-CRUD describe checks + token-shape validation + per-operation field allowlists at every guest entry point. v2.1.0 layers DocGenFlsGuard per-field guards on top.
  - **Verifier:** DocGenAuthenticatorController.verifyDocument returns `List<VerificationResult>` (was LIMIT 1) so multi-signer documents render the complete audit trail.
- 

*File maintained at docs/appexchange/v2.1.0/*

*Listing\_Platform\_Technology.md so future re-submissions can pull this verbatim or update in place.*